



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 2 (10%) – Transaction Management

Name	:	Goe Jie Ying	Tan Yi Ya
Matric Id	:	A23CS0224	A23CS0181
Section	:	02	

Instructions:

This assignment evaluates your ability to apply the concept of serializability, differentiate between serial and non-serial transaction schedules and implement locking techniques to manage simultaneous transactions.

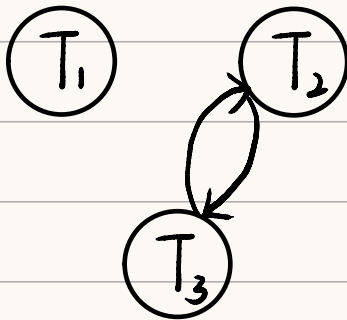
Academic Integrity

All students must work in a group of **TWO(2)**. The use of AI-generated tools or sharing of answers between students **is strictly prohibited** and will result in **zero marks**. You are required to complete the answers in **handwritten form**. Please take **clear screenshots** of your answers and submit them in **PDF format**.

Assignment 2

Question 1

i)

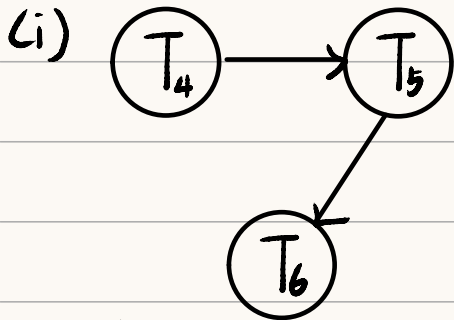


ii) The precedence graph shows that the schedule is non-conflict serializable. There are two conflicts: $T_2 \rightarrow T_3$ and $T_3 \rightarrow T_2$, which form a cycle. Because of this cycle, the schedule cannot be transformed into any equivalent serial schedule.

iii)	Time	T1	T2	T3
	t_1	Begin Transaction		
	t_2	s_lock(X)		
	t_3	read_item(X)	Begin Transaction	
	t_4	unlock(X)	x_lock(M)	Begin Transaction
	t_5	s_lock(Y)	read_item(M)	x_lock(N)
	t_6	read_item(Y)	s_lock(N)	read_item(N)
	t_7	unlock(Y)	WAIT	s_lock(M)
	t_8		...	WAIT
	...		...	...

iv) Deadlock, caused by circular waiting between T2 and T3. They are both holding x-lock and requesting s-lock for same item. Starvation may still occurs when the transaction is perpetually delayed.

Question 2



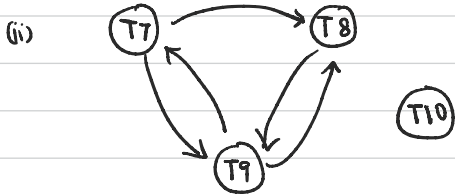
(ii) The precedence graph shows that the schedule is conflict serializable because it contains no cycle and can be reordered into a serial schedule without violating any conflict.

(iii)

	T4	T5	T6
	Begin transaction		
	α -lock(P)		
	read-item(P)	Begin transaction	
	S-lock(Q)	S-lock(Q)	
	read-item(Q)	read-item(Q)	Begin transaction
	$P = (P * 5) + Q$	S-lock(R)	S-lock(R)
	write-item(P)	read-item(R)	read-item(R)
	unlock(P) & (Q)	WAIT	WAIT
time		α -lock(Q)	WAIT
↓		$Q = Q + R$	WAIT
		write-item(Q)	WAIT
		unlock(Q) & (R)	WAIT
			α -lock(R)
			$R = R - 100$
			write-item(R)
			unlock(R)

Question 3

(a) (i) A non-serial schedule. T7, T8, T9 and T10 executes concurrently without waiting for the first transaction to finish before starting the next. It allow multiple transaction to interleave before it completely end.



(iii) It is non conflict serializable, since the precedence graph contain cycle at $T7 \rightarrow T9, T9 \rightarrow T8$ and $T8 \rightarrow T7$.

(iv) Lost update. At t_6 and t_7 , T9 and T8 both write X. T8 overwrite T9's write without reading T9's updated value.

(b) (i)

T11	T12	T13	T14
begin transaction			
X-lock(A)	begin transaction		
read(A)	X-lock(B)		
$A = A + 200$	read(B)	begin transaction	
write(A)	$B = B - 50$	X-lock(A)	begin transaction
unlock(A)	write(B)	wait	X-lock(C)
	unlock(B)	wait	read(C)
		read(A)	$C = C + 10$
commit		$A = A - 30$	write(C)
	X-lock(A)	write(A)	unlock(C)
	wait	unlock(A)	
	read(A)		
	$A = A * 2$		
	write(A)		
	commit/unlock(A)	commit	commit

Question 3

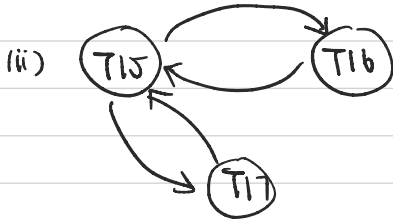
b (ii) Deadlock, starvation and livelock might occur.

(iii)	T11	T12	T13	T14
	begin transaction			
	X-lock (A)	begin transaction		
	read (A)	X-lock (B)	begin transaction	
	$A = A + 200$	read (B)	X-lock (A)	begin transaction
	write (A)	$B = B - 50$	wait	X-lock (C)
	unlock (A)	write (B)	wait	read (C)
			read (A)	$C = C + 10$
	commit		$A = A - 30$	write (C)
		X-lock (A)	write (A)	unlock (C)
		wait	unlock (A)	
		read (A)		
		$A = A * 2$		
		write (A)		
		commit / unlock (A, B)	commit	commit

(v) Cascading Rollback and deadlock

Question 4

- (i) non-serial schedule. T_{15} , T_{16} , T_{17} executes concurrently without waiting for the first transaction to finish before starting the next. It allows multiple transactions to be interleaved.



- (iii) Non conflict serializable. There is a loop in the precedence graph.

- (iv) Lost updates. For example, T_{16} writes K at t_5 and T_{15} writes again at t_7 . Uncommitted Dependency (Dirty Read). For example, T_{17} reads J at t_4 , but at the same time T_{15} writes J .

(v)	T_{15}	T_{16}	T_{17}
	begin transaction		
	X-lock(J)	begin transaction	
	read(J)	X-lock(K)	
	$J = J + 10$	read(K)	begin transaction
	write(J)	$K = K - 5$	X-lock(J)
	X-lock(K)	write(K)	wait
	wait	commit/unlock(K)	wait
	read(K)		wait
	$K = K + 20$		wait
	write(K)		wait
	commit/unlock(J,K)		wait
			read(J)
			$J = J - 15$
			write(J)
			commit/unlock(J)